

Manual Q&A operator_guide 프로토 발표 정리

1. 프로토 목표

Manual Q&A Agent 프로토의 목표는 플레이어가 게임 중 질문을 했을 때 operator_guide Agent가 CSV 기반 매뉴얼 데이터를 조회해서 답변을 반환하는 구조를 검증하는 것입니다.

플레이어 질문

- > operator_guide 선택
- > 질문 유형 분류
- > CSV 매뉴얼 조회
- > final_answer 반환
- > Unreal UI에 표시

프로토에서는 아직 LLM, PostgreSQL, pgvector, Embedding, 실시간 player_state 분석은 사용하지 않습니다.

2. Unreal 쪽 역할

Unreal UI 또는 Front는 플레이어가 입력한 질문을 서버에 JSON 형태로 보냅니다.

Input 예시

```
{
  "type": "agent.request",
  "client_id": "unreal-ui-001",
  "agent": "operator_guide",
  "payload": {
    "question": "기어 만들려면 뭐가 필요해?"
  }
}
```

Unreal이 보내는 주요 값

```
client_id
-> 어떤 Unreal UI 또는 Front에서 보낸 요청인지 구분하는 값

agent
-> 현재 프로토에서는 operator_guide로 보냄

payload.question
-> 플레이어가 실제로 입력한 질문
```

프로토에서는 request_id, session_id를 생략할 수 있습니다. 서버가 요청 처리 중 request_id를 만들 수 있고, session_id는 아직 대화 기억을 적극적으로 사용하지 않기 때문에 null이어도 됩니다.

3. AI / Backend 쪽 역할

서버는 Unreal에서 받은 질문을 AgentPipeline으로 넘깁니다.

프로토 처리 흐름

```
Unreal UI / Front
-> AgentPipeline
-> Orchestrator Router
-> operator_guide Agent
-> leaf agent router
-> ManualQAService
-> ManualQAQuestionClassifier
-> CsvManualQARepository
-> ManualQAResponseBuilder
-> agent.response JSON
```

각 단계 역할

AgentPipeline

-> 요청 ID, 세션 ID, 클라이언트 ID 같은 실행 컨텍스트를 정리합니다.

Orchestrator Router

-> 여러 Agent 중 어떤 Agent가 맞는지 판단합니다.
-> 장비, 자원, 레시피, 트러블슈팅 질문이면 operator_guide를 선택합니다.

operator_guide Agent

-> 게임 매뉴얼 Q&A를 담당하는 Agent입니다.

leaf agent router

-> 질문 성격에 따라 machine_help, recipe_explainer, troubleshooter 중 하나로 나눕니다.

ManualQAService

-> 실제 CSV 기반 답변을 만드는 중심 서비스입니다.

ManualQAQuestionClassifier

-> 질문을 equipment_question, resource_question, recipe_question, troubleshooting_question, unknown_question으로 분류합니다.

CsvManualQARepository

-> 5개 CSV 파일에서 필요한 데이터를 조회합니다.

ManualQAResponseBuilder

-> 조회 결과를 final_answer, sources, recommended_actions 형태로 조립합니다.

4. 매뉴얼 문서 관리 방식

매뉴얼 문서는 자유 텍스트 FAQ로만 관리하지 않고, 데이터 스키마를 기준으로 나눠 관리합니다.

현재 프로토 CSV는 5개입니다.

```
equipment.csv
-> 제련기, 채굴기, 조립기, 발전기 같은 장비 정보

resources.csv
-> 철광석, 철괴, 구리광석, 구리괴 같은 자원 정보

recipes.csv
-> 자원을 어떤 장비에서 어떤 결과물로 만드는지에 대한 제작 공정

troubleshooting_rules.csv
-> 장비가 멈췄을 때, 전력이 부족할 때 같은 문제 해결 규칙

action_policy.csv
-> 플레이어에게 추천할 확인 행동
```

Unreal에서 게임 데이터가 정해진 스키마에 맞게 제공되면, 프로토 단계에서는 그 데이터를 CSV로 정리해 매뉴얼 지식 베이스로 사용합니다. operator_guide는 플레이어 질문을 분류한 뒤 관련 CSV를 조회해서 답변을 반환합니다.

5. Output 구조

AI는 Unreal에 agent.response JSON을 반환합니다.

Output 예시

```
{
  "type": "agent.response",
  "request_id": "server_generated_uuid",
  "session_id": null,
  "client_id": "unreal-ui-001",
  "agent": "operator_guide",
  "payload": {
    "final_answer": "좋아요. 기어를 만들려면 철괴 2개가 필요합니다. 조립기에서 제작하고, 흐름은 철광석 채굴 > 철괴 제련 > 조립기에서 기어 제작 순서로 보면 됩니다.",
    "actions": [],
    "metadata": {
      "question": "기어 만들려면 뭐가 필요해?",
      "question_type": "recipe_question",
      "sources": [
        {
          "doc_id": "recipe_gear",
          "type": "recipe",
          "title": "기어 제작 공정"
        }
      ]
    },
    "recommended_actions": [
      {
        "action_id": "action_explain_recipe_requirements",
        "label": "레시피 요구사항 설명",
        "priority": 1
      }
    ],
    "confidence": "high"
  }
},
"streams": []
}
```

Unreal 화면에는 아래 값만 표시하면 됩니다.

```
payload.final_answer
```

metadata는 화면 표시용이라기보다는 출처, 추천 행동, 디버깅, UI 보조 정보로 사용할 수 있습니다.

6. Unknown 질문 처리

CSV 매뉴얼 범위 밖 질문은 억지로 답하지 않고 안내형 답변을 반환합니다.

Unknown 질문 예시

우주 엘리베이터는 어떻게 업그레이드해?

응답 방향

현재 매뉴얼에서는 이 질문에 대한 정보를 찾을 수 없습니다.
장비, 자원, 레시피, 생산 문제와 관련된 질문을 해주세요.

프로토에서는 모르는 질문에 대해 환각 답변을 만들지 않는 것이 목표입니다.

7. 프로토에서 한 것

1. operator_guide Agent를 Manual Q&A 용도로 연결
2. 질문 유형 분류 구조 구현
3. 5개 CSV 기반 매뉴얼 조회 구조 구현
4. 장비, 자원, 레시피, 트러블슈팅, unknown 질문 처리
5. final_answer 중심 응답 구조 정리
6. Unreal이 받을 input/output JSON 형태 정리
7. 프로토 흐름을 확인할 수 있는 Jupyter Notebook 작성
8. Mermaid 그래프로 Agent 흐름 시각화
9. 사용자용 문서와 시스템용 문서 작성

8. 프로토에서 아직 하지 않은 것

실시간 player_state 분석
PostgreSQL 연동
pgvector 기반 문서 검색
Embedding 생성
LLM 기반 답변 생성
대화 기억 기반 session 관리
Unreal action 실제 실행

9. 최종 구조로 고도화 방향

프로토

-> CSV 기반 정적 매뉴얼 답변

알파

-> Unreal player_state 일부 전달

-> 현재 선택 장비, 전력, 입력 자원 상태를 반영

베타

-> PostgreSQL 구조화 데이터로 전환

-> 장비, 자원, 레시피, 트러블슈팅 데이터를 DB에서 조회

최종

-> pgvector + Embedding으로 매뉴얼 문서 검색

-> LLM이 검색 근거와 player_state를 보고 자연스러운 답변 생성

10. 발표용 핵심 문장

이번 프로토는 operator_guide Agent가 Unreal에서 받은 플레이어 질문을 CSV 기반 매뉴얼 데이터와 연결해 답변하는 구조를 검증한 단계입니다.

최종 단계에서는 이 구조를 PostgreSQL, pgvector, LLM, player_state 분석으로 확장해 현재 상황에 맞는 AI 안내 에이전트로 고도화할 예정입니다.